ENGENHARIA E PROCESSOS DE SOFTWARE

Fases do Ciclo de Vida do Software

Prof. Manoel Dantas Macedo Filho

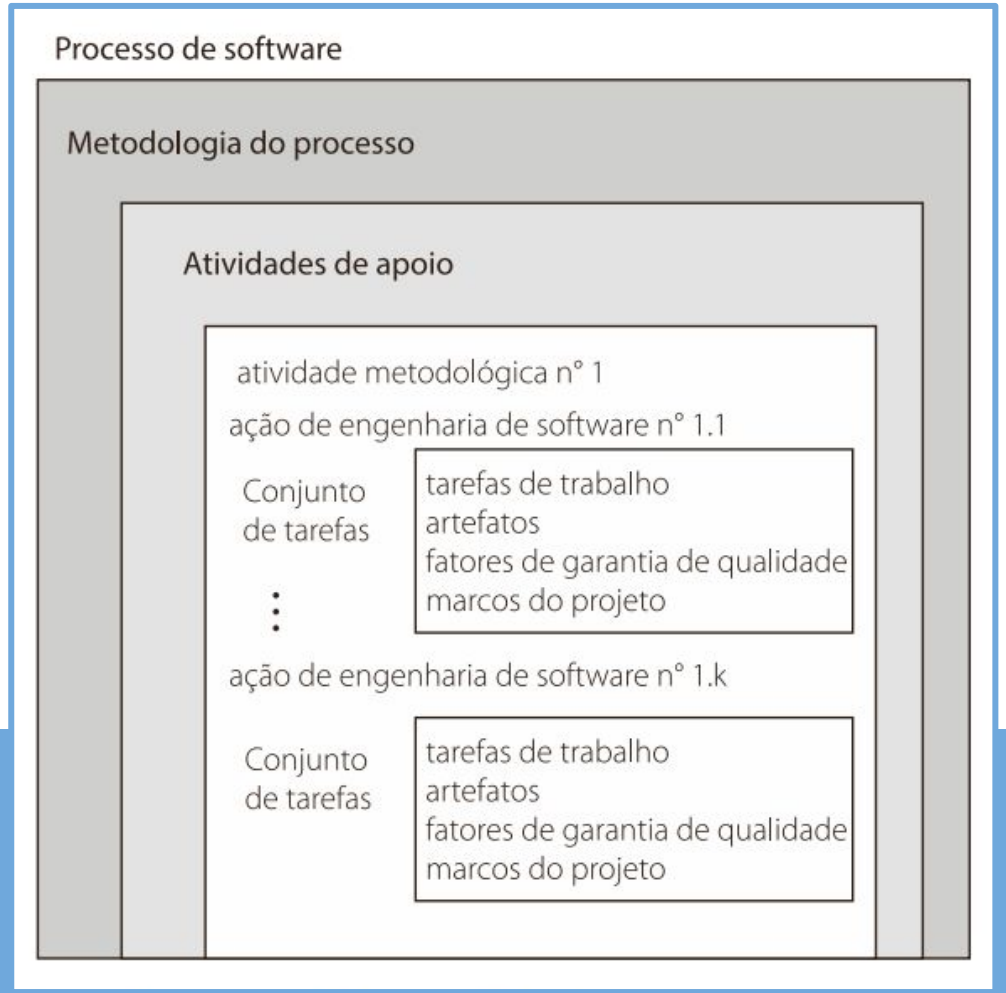
Introdução:

O desenvolvimento de software exige planejamento, organização e execução cuidadosa. O ciclo de vida do software é um modelo que descreve as etapas fundamentais desde a concepção até a desativação de um sistema. Entender essas fases é essencial para garantir qualidade, eficiência e atendimento às necessidades do cliente.

Conceito

O **ciclo de vida de software** é o conjunto de processos, atividades e tarefas envolvidas no desenvolvimento, operação e manutenção de um sistema. Vai desde a definição dos requisitos até a retirada do sistema de uso. Ele serve como guia para conduzir o projeto de forma estruturada e controlada.

AGILE
DEVELOPMENT



Um fluxo de processo, como apresentado na Figura 1 a seguir, descreve como são organizadas as atividades metodológicas, bem como as ações e as tarefas que ocorrem dentro de cada atividade em relação à sequência e ao tempo…

FASES DO CICLO DE VIDA DO DESENVOLVIMENTO DE SOFTWARE
Planejamento
Análise de requisitos
Design do sistema
Implementação
Testes
Implantação
Manutenção

A importância das fases

Cada fase do ciclo de vida tem seu papel estratégico. Elas garantem:

- Identificação clara dos requisitos do cliente
- Planejamento técnico e de tempo
- Produção com qualidade
- Testes adequados
- Atualizações contínuas conforme mudanças do contexto ou das necessidades

ATIVIDADE FUNDAMENTAL 1 – Especificação de Software

Nesta fase, são levantadas as **necessidades do cliente**. É aqui que se entende o problema a ser resolvido. Essa etapa envolve:

- Estudo de viabilidade
- Elicitação e análise de requisitos
- Documentação dos requisitos do usuário e do sistema

ATIVIDADE FUNDAMENTAL 2 – Projeto e Implementação

Converte-se a especificação em um produto funcional. Essa fase é dividida em:

- Projeto de arquitetura e detalhado
- Codificação dos módulos
- Integração entre componentes
  O foco está na organização lógica e física do sistema.

ATIVIDADE FUNDAMENTAL 3 – Validação do Software

Aqui testamos se o software **faz o que foi solicitado**. Os testes são realizados em etapas:

- Teste de componente (módulo por módulo)
- Teste de sistema (como um todo)
- Teste de aceitação (com o cliente)

  Validação é a garantia de conformidade com os requisitos.

ATIVIDADE FUNDAMENTAL 4 – Evolução do Software

Após a entrega, o sistema continua sendo atualizado. Isso inclui:

- Correções de falhas
- Melhorias de desempenho
- Adaptações a novos ambientes

A evolução garante que o software continue útil ao longo do tempo.

AGILE
DEVELOPMENT
321

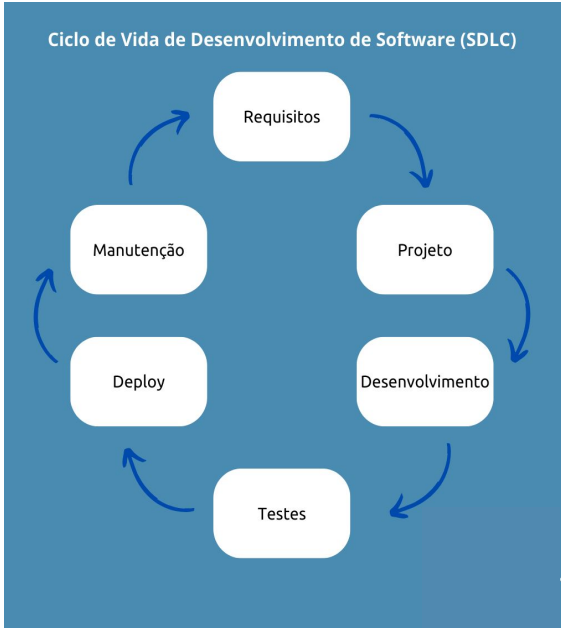
Atividades Metodológicas e Apoio

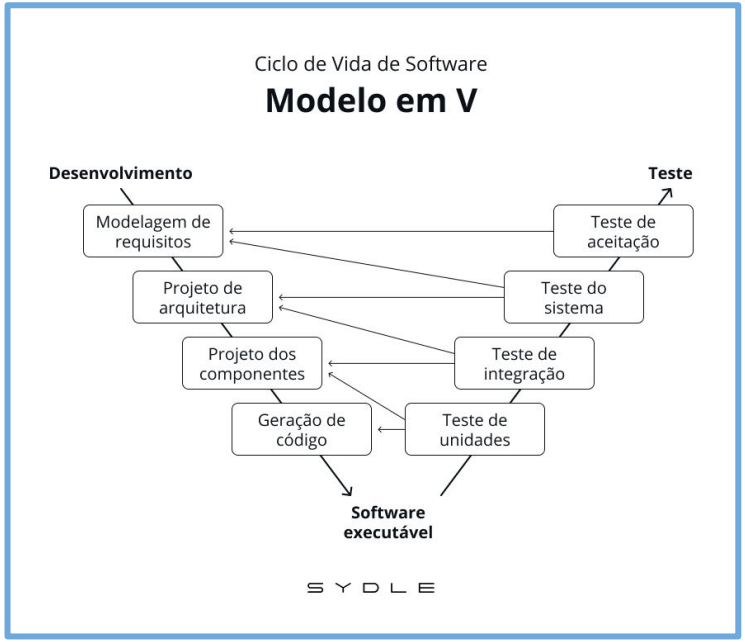
Além das fases principais, há **atividades de apoio**:

- Comunicação com o cliente
- Planejamento do projeto
- Modelagem do sistema
- Construção e entrega
- Gestão de riscos e controle de qualidade
  Essas ações fortalecem o processo e reduzem falhas.

Considerações

O ciclo de vida do software **não é fixo**. Ele pode ser adaptado ao contexto do projeto. Modelos como Cascata, Espiral ou Prototipagem diferem na ordem ou intensidade de cada fase. O mais importante é garantir a qualidade, o prazo e a satisfação do cliente.





Curiosidades

- 🧠 Segundo Ian Sommerville, “Nenhum sistema de software está completo ao ser entregue”. Ele precisa evoluir junto com as mudanças tecnológicas e demandas sociais.

- 💡 O modelo **RAD (Rapid Application Development)** encurta o ciclo de vida para projetos que exigem entrega rápida, mas não é indicado para sistemas complexos.

- 🎮 Jogos, redes sociais e sistemas bancários seguem ciclos de vida diferentes, mas todos precisam das fases fundamentais de engenharia de software.

CITAÇÃO!

"Não há bala de prata – nenhuma tecnologia ou prática da engenharia de software sozinha irá dobrar a produtividade em um ano ou dois."

— Fred Brooks, em No Silver Bullet – Essence and Accidents of Software Engineering (1986)

Questões Objetivas

1. Qual das alternativas representa corretamente uma **fase fundamental** do ciclo de vida de software?

A) Especificação de requisitos

B) Instalação do sistema operacional

C) Compra de servidores

D) Treinamento de equipe

E) Divulgação publicitária

F) Descarte de backups

Questões Objetivas

2. A fase de **validação de software** tem como principal objetivo:

A) Elaborar a arquitetura do banco de dados

B) Avaliar se o sistema atende às especificações e ao cliente

C) Atualizar as tecnologias utilizadas no projeto

D) Negociar prazos com o cliente

E) Garantir que a linguagem de programação está atualizada

F) Medir o consumo energético dos servidores

Questões Objetivas

3. Sobre a fase de **evolução do software**, é correto afirmar que:

A) É desnecessária se o sistema estiver bem documentado

B) Só ocorre após o ciclo de vida terminar

C) Envolve atualizações, correções e melhorias contínuas

D) Serve apenas para avaliar o desempenho inicial

E) Tem foco exclusivo em testes de segurança

F) Ocorre antes da validação do sistema

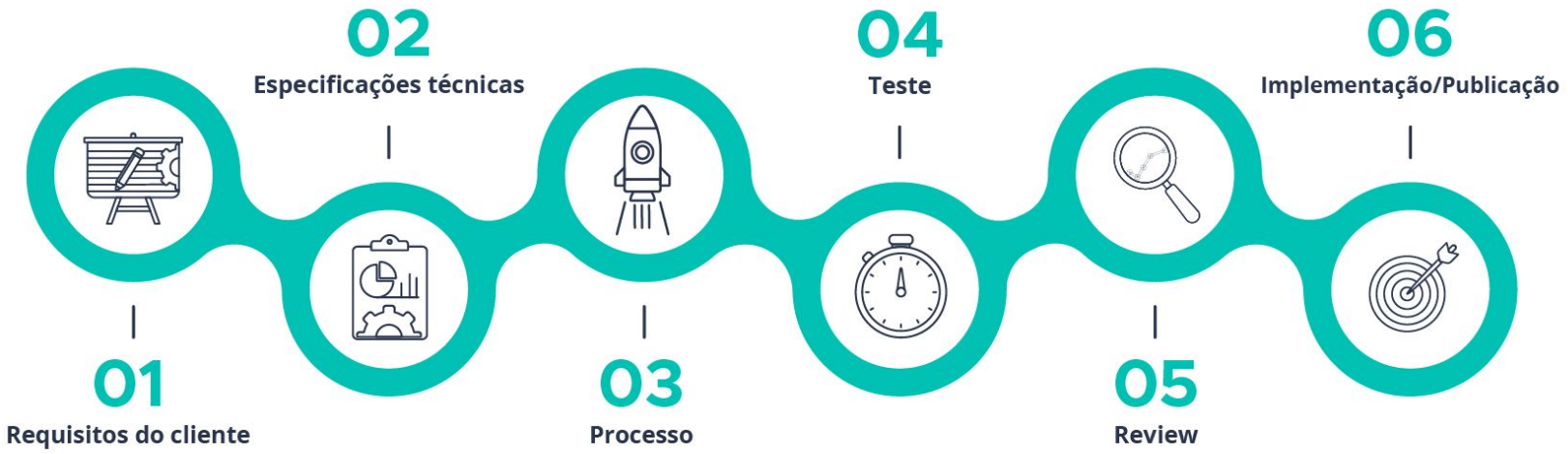
ALGUNS MÉTODOS...

ALGUNS MÉTODOS...

Os **processos de software** são as atividades envolvidas na produção de um produto de software. Eles são inerentemente complexos e dependem significativamente da criatividade humana, o que tem dificultado tentativas de automatização. O objetivo principal da utilização desses processos é entregar softwares de qualidade, dentro do prazo esperado e, crucialmente, garantir a satisfação do cliente/usuário. Embora existam muitos processos distintos, todos compartilham **atividades fundamentais** como especificação, projeto, implementação, validação e evolução do software.

ALGUNS MÉTODOS...

A evolução rápida da economia, serviços concorrentes e novos produtos exige que os negócios respondam com igual velocidade para aproveitar novas oportunidades. Como o software é parte integrante das operações de negócio, é essencial desenvolvê-lo rapidamente. Pensando nisso, em 2001, Kent Beck e outros 16 desenvolvedores assinaram o **"Manifesto para o desenvolvimento Ágil de Software"**, dando origem a processos que visam criar software útil rapidamente. Esses são geralmente processos **iterativos**, onde especificação, projeto, desenvolvimento e teste são intercalados.


01
Requisitos do cliente
02
Especificações técnicas
03
Processo
04
Teste
05
Review
06
Implementação/Publicação

artia

CITAÇÃO!

"Indivíduos e interações
acima de processos e ferramentas."

— Manifesto Ágil (2001)

ALGUNS MÉTODOS…

As abordagens de desenvolvimento rápido de software compartilham características essenciais. O software não é desenvolvido e disponibilizado integralmente, mas sim em uma **série de incrementos**, cada um adicionando uma nova funcionalidade ao sistema. Nesses métodos, os processos de especificação, projeto e implementação são concorrentes, e as interfaces com o usuário são desenvolvidas usando um sistema interativo.

ALGUNS MÉTODOS…

Para garantir a agilidade, os **métodos ágeis** seguem princípios fundamentais. A **maior prioridade é satisfazer o cliente**, acolher bem as alterações (mesmo que atrasem o projeto), e fazer **entregas frequentes de software em funcionamento**. Outros princípios incluem a colaboração entre a equipe comercial e os desenvolvedores, manter a equipe motivada através de conversas abertas e presenciais, e considerar o software em funcionamento como a principal medida de progresso. A excelência técnica, simplicidade, e a emergência de melhores arquiteturas e requisitos de equipes auto-organizadas também são cruciais. O envolvimento profundo do cliente, a entrega incremental, a valorização das pessoas sobre o processo, a aceitação de mudanças e a manutenção da simplicidade são princípios destacados.

ALGUNS MÉTODOS...

Entre as metodologias ágeis mais usadas, destacam-se XP, Scrum e OpenUP. O **Extreme Programming (XP)** é o método ágil mais conhecido. No XP, os requisitos são expressos como cenários implementados diretamente como tarefas. Os programadores trabalham em pares e desenvolvem testes antes mesmo de escrever o código. O XP é fundamentado em cinco **valores essenciais**: Comunicação, Simplicidade, Feedback, Coragem e Respeito.

ALGUNS MÉTODOS…

O processo XP envolve etapas de planejamento, projeto, codificação e testes, com uma abordagem orientada a objetos. Ele apoia o desenvolvimento incremental através de **pequenos e frequentes lançamentos do sistema**, baseados nos cenários dos clientes. Os clientes são ativamente envolvidos na especificação e priorização dos requisitos, participando das discussões e desenvolvendo "cartões de histórias" que representam suas necessidades. Ao contrário da engenharia de software tradicional, o XP descarta a premissa de prever futuras mudanças, considerando que projetar para mudanças que podem nunca ocorrer é um esforço inútil.

ALGUNS MÉTODOS…

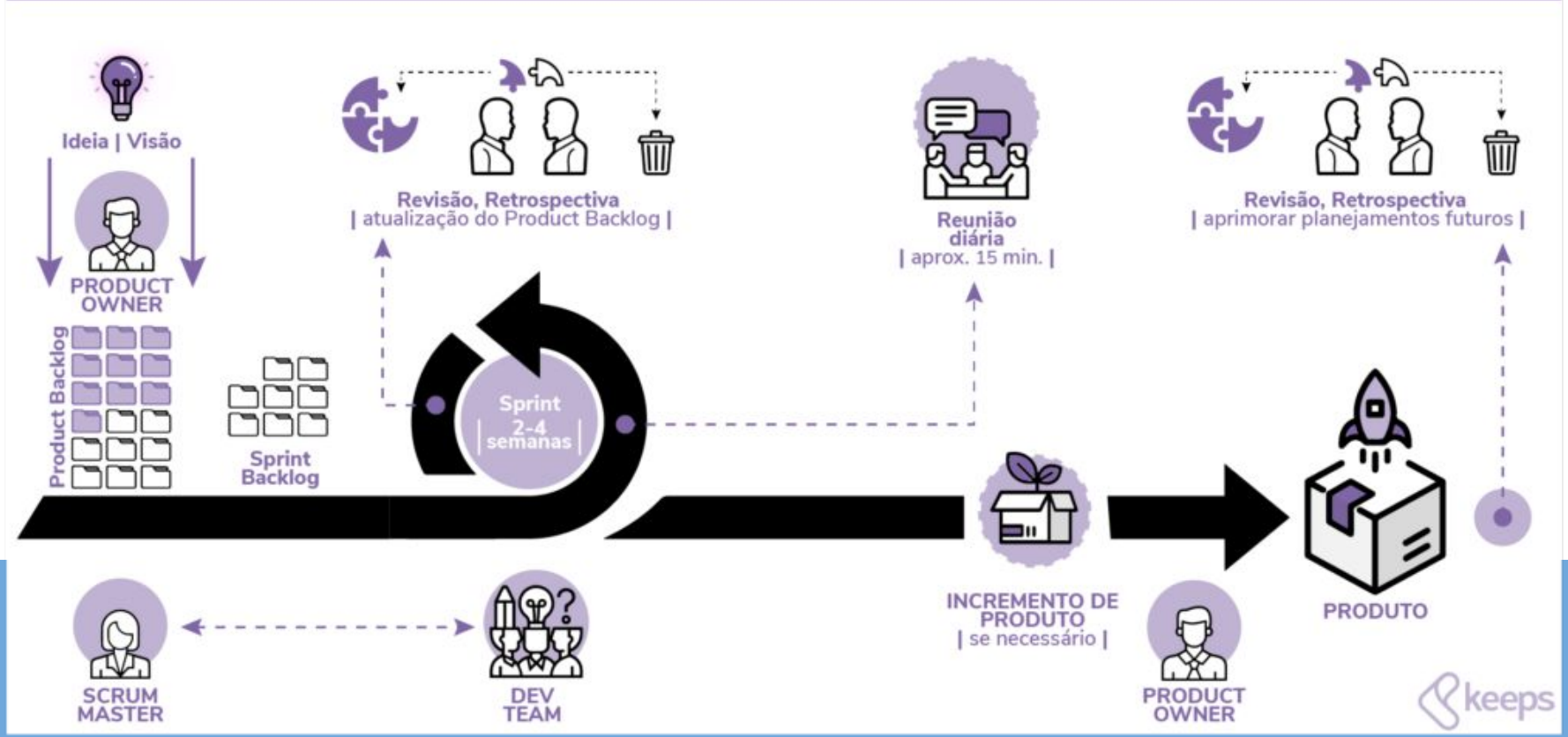
Dentro de uma equipe XP, há **papéis específicos** que visam otimizar a contribuição de cada membro para o sucesso do projeto e da equipe. Estes incluem Executivos (definem escopo e alinham com a organização), Gerentes de Projeto (ponte entre equipe, clientes e fornecedores), Gerentes de Produto (asseguram que o produto se desenvolva conforme o projeto), Arquitetos (auxiliam desenvolvedores e na criação de testes de arquitetura), Analistas de Teste (ajudam a escrever testes), Designers de Interação (criam interfaces próximas aos clientes), Programadores (trabalham em pares e criam funcionalidades), Recursos Humanos (recrutam e selecionam a equipe) e Redatores Técnicos (mantêm a documentação). Os **Usuários** também têm participação valiosa desde o início, especialmente se dominam o negócio.

ALGUNS MÉTODOS…

O **Scrum** é um framework para resolver problemas complexos e adaptativos, entregando produtos de alto valor. É leve, simples de entender e desafiador de dominar. No Scrum, os projetos são divididos em ciclos mensais chamados **sprints**, que representam um "time box" para um conjunto de atividades como requisitos, análise, projeto, evolução e entrega. As atividades são guiadas por um **Registro Pendente de Trabalho (backlog)**, uma lista priorizada de requisitos ou funcionalidades.

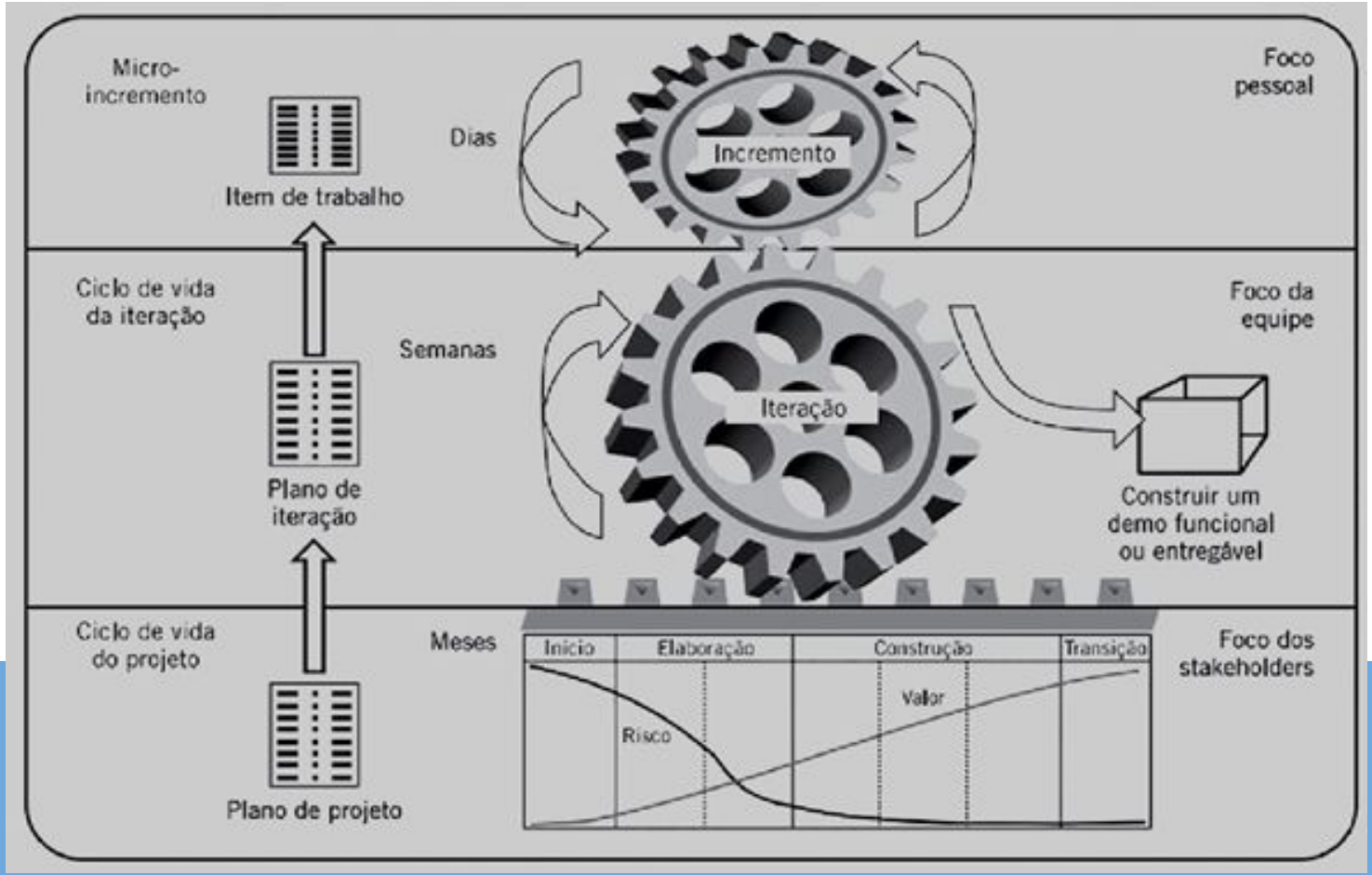
Um princípio central do Scrum é a **transparência**. Reuniões Scrum diárias são realizadas para que a equipe monitore o progresso, discuta o que foi realizado, dificuldades e planos futuros. O progresso é monitorado através de um **release burndown chart**, um gráfico atualizado ao final de cada sprint para mostrar o trabalho restante. O **Quadro Scrum** é uma ferramenta visual para organizar o backlog e acompanhar o andamento das tarefas, podendo ser um simples quadro branco ou um software especializado.

ESQUEMA DE FUNCIONAMENTO DO SCRUM



ALGUNS MÉTODOS…

O **OpenUP** é uma metodologia que segue o processo unificado de engenharia de software, mas com uma **cultura aberta e enxuta**, focando na natureza colaborativa do desenvolvimento. Ele combina elementos de metodologias ágeis como Scrum e XP com a arquitetura do processo unificado. Seus **princípios** incluem colaborar para alinhar interesses, balancear prioridades concorrentes para maximizar o valor dos stakeholders, focar na arquitetura para minimizar riscos, e envolver os stakeholders para obter feedback contínuo. O OpenUP é baseado em um **ciclo iterativo e incremental**, com foco na criação de um modelo ágil e enxuto. Possui 4 fases principais em seu ciclo de vida: **Iniciação** (análise de negócios e requisitos), **Elaboração** (desenvolvimento da análise arquitetural), **Construção** (implementação da solução, testes e integração), e **Transição** (implantação do release, teste beta, treinamento do usuário e conversão de dados). Durante a Construção, são feitas entregas incrementais das funcionalidades aos stakeholders. Os **papéis** no OpenUP incluem Analista, Arquiteto, Desenvolvedores, Gerente de Projeto, Stakeholders e Testador.


Micro-
incremento
Item de trabalho
Dias
Incremento
Foco
pessoal
Ciclo de vida
da iteração
Plano de
iteração
Semanas
Iteração
Foco da
equipe
Construir um
demo funcional
ou entregável
Ciclo de vida
do projeto
Plano de projeto
Meses
Início
Elaboração
Construção
Transição
Risco
Valor
Foco dos
stakeholders

ALGUNS MÉTODOS…

Apesar dos benefícios, a adoção de metodologias ágeis pode falhar devido a diversos fatores. As **principais causas de insucesso** incluem filosofias ou culturas empresariais que conflitam com os valores ágeis, falta de suporte gerencial para as mudanças, experiência ou treinamento insuficientes no novo processo, e a falta de comprometimento ou até boicote da própria equipe.

Por fim, vale ressaltar que Scrum e XP, apesar de serem metodologias distintas, compartilham algumas **semelhanças conceituais** em suas práticas. Por exemplo, o "Sprint" no Scrum é similar à "Iteração" no XP, e o "Daily Scrum" é análogo à "Stand-up meeting".

AGILE
DEVELOPMENT

CITAÇÃO!

"Melhorar é mudar;
ser perfeito é mudar com frequência."

— Winston Churchill

Obrigado!

[manoelmacedo@yahoo.com]

Fontes:

Wikipédia, www.sydle.com, Google Imagens, USP, agilealliance.org, www.atlassian.com, aws.amazon.com, scrumguides.org, medium.com, repositorio.utfpr.edu.br, educapes.capes.gov.br, Microsoft, Pinterest, agilemanifesto.org